MHEIClosed: An Efficient Algorithm for Mining Closed High-Efficiency Itemsets

Linh T. T. Nguyen[1,2,3](✉), Thang G. Phung[1,2], Vinh Q. Pham[1,2], and Loan T. T. Nguyen[1,2](✉)

[1] International University, Ho Chi Minh City, Vietnam
{linhntt25,vinhpq25}@mp.hcmiu.edu.vn, {pgthang, nttloan}@hcmiu.edu.vn
[2] Vietnam National University, Ho Chi Minh City, Vietnam
[3] Thu Duc College of Economics and Technology, Ho Chi Minh City, Vietnam

Abstract. High-Efficiency Itemset Mining (HEIM) is an emerging extension of utility-based mining that considers utility and investment, addressing key limitations of High Utility Itemset Mining (HUIM). However, existing HEIM methods often produce large and redundant result sets, complicating interpretation and post-analysis. To address this, we propose Closed High-Efficiency Itemset Mining **(CHEIM)**, which focuses on extracting concise and non-redundant patterns while preserving essential utility and investment information. This paper presents **MHEIClosed**, a novel algorithm for efficiently mining closed high-efficiency itemsets. MHEIClosed incorporates advanced techniques from closed itemset and HEIM approaches, including pruning strategies based on tighter upper bounds (*ssef* and *slef*), transaction merging, and efficient closure checking. Extensive experiments on several real-world datasets demonstrate that MHEIClosed significantly outperforms the state-of-the-art HEPMClosed algorithm in terms of runtime and memory consumption, particularly on dense or large-scale datasets. These results validate MHEIClosed's scalability and effectiveness in mining meaningful patterns, making it well-suited for practical data mining applications.

Keywords: High-efficiency itemset · closed pattern mining · pruning strategies

1 Introduction

This section introduces an overview of the field and research issues in this paper.

High Utility Itemset Mining (HUIM) is a key area in data mining, applied in business and supply chain management. Unlike Frequent Itemset Mining (FIM) [1], HUIM lacks the anti-monotonicity property, making HUI discovery more complex and costly. HUIM often produces redundant and overlapping patterns, which complicates analysis and other tasks, such as association rule mining. Closed High-Utility Itemsets (CHUIs) were introduced to address this issue, with algorithms such as CHUI-Miner [2], EFIM-Closed [3], CLS-Miner [4], and iEFIM-Closed [5]. However, HUIM ignores investment, which is crucial in business. High-Efficiency Itemset Mining (HEIM) was proposed [6] to

© The Author(s), under exclusive license to Springer Nature Singapore Pte Ltd. 2026
T. T. Quan et al. (Eds.): MIWAI 2025, LNAI 16354, pp. 345–357, 2026.
https://doi.org/10.1007/978-981-95-4960-3_28

address this, focusing on the utility-to-investment ratio. HEPM, HEPMiner [6], and MHEI [7] are current algorithms developed for mining High-Efficiency Itemsets (HEIs).

In this study, we introduce a new method for HEIM: Closed High-Efficiency Itemset Mining (CHEIM). This method decreases the number of patterns while preserving safety information through the closure property, thus avoiding redundancy. We also present a new algorithm called MHEIClosed for CHEIM, developed based on MHEI and EFIM-Closed. This algorithm uses advanced pruning techniques to improve runtime and reduce memory usage.

Section 2 reviews related work on HUIM, CHUIM, and HEIM and their contributions. Section 3 defines the CHEIM problem and presents the core definitions and properties used in the algorithm. Section 4 details the MHEIClosed algorithm. Section 5 presents experimental results. Finally, Sect. 6 summarizes the study and discusses future work.

2 Related Work

HEIM has been extensively studied as an extension of Frequent Itemset Mining (FIM). However, since utility does not satisfy the Downward Closure Property (DCP) [1], effectively pruning the search space is more challenging in HUIM. To address this, algorithms have proposed upper bounds for pruning unpromising itemsets.

In FIM, classic algorithms such as Apriori [8] and FP-Growth [9] use support that satisfies DCP for efficient mining. These methods have been successfully applied in various fields, such as market basket analysis and recommender systems. However, with HUIM, only support is not complete, necessitating the integration of utility measures.

One of the earliest and well-known methods in HUIM is the Two-Phase algorithm [10]. It works in two steps: first, it uses the TWU to filter itemsets below a minimum utility threshold (minutil) and select promising candidates. Next, it rescans the database to compute the actual utility of these candidates, retaining only those that meet or exceed minutil. However, this method is memory-heavy for large or dense datasets. HUI-Miner [11] uses a utility-list structure and a tighter upper bound to improve efficiency, making it faster and less memory-consuming than Two-Phase. Although the utility-list structure boosts mining performance, its joins are costly. In 2014, FHM [12] introduced the Estimated Utility Co-occurrence Structure (EUCS) to store TWU values for 2-itemsets, enabling early pruning and reducing joins. HUP-Miner [13] later improved this with Look-Ahead Pruning (LA-Pruning), which tightened the remaining utility and enhanced pruning efficiency. In 2018, ULB-Miner [14] used a Utility-List Buffer (ULB) to store and release utility-list segments temporarily. In 2017, HMiner [15] was proposed with a complete framework, integrating TWU, EUCS, and LA-Pruning to prune unpromising branches early, minimise joins, optimize memory, and achieve better performance than previous approaches.

Most HUIM algorithms use TWU for pruning, utility-list for storage, and multiple scans. In 2017, Zida et al. proposed EFIM [16], introducing sub-tree and local utility upper bounds with a fast counting method, database projection, and transaction merging. However, EFIM remained inefficient on sparse data due to unnecessary scans. In 2019, Nguyen et al. proposed iMEFIM/MEFIM [17], using P-set structures to track item positions and reduce redundant scans efficiently.

HUIM algorithms often generate many overlapping HUIs, complicating analysis. To address this, CHUI-Miner [2] used the EU-List to directly mine closed HUIs (CHUIs). EFIM-Closed [3] extended EFIM for CHUI mining, reducing output size. CLS-Miner [4] introduced advanced pruning strategies, while iEFIM-Closed [5] applied P-set structures in dynamic profit databases, emphasizing the value of compact and efficient CHUI results.

Frequent itemset mining focuses on occurrence frequency, overlooking rare but highly profitable patterns. Similarly, HUIM emphasizes utility while neglecting investment. Zhang et al. introduced HEPM, which defines high-efficiency itemsets based on the utility-to-investment ratio, proposing two algorithms: HEPM (Two-Phase, memory-intensive) and HEPMiner, which improves pruning using the Efficiency-List, EESC, and upper bounds (SEUB, SEUBW) [6]. In 2024, Huynh et al. proposed MHEI [7], integrating transaction merging, transaction ID tracking, and two tighter upper bounds: ssef and slef to enhance pruning and reduce the search space.

3 Definitions and Properties

Closed High-Efficiency Itemset Mining (CHEIM) aims to find itemsets whose efficiency (profitability-to-investment ratio) meets or exceeds a user-defined threshold. Inputs include a transaction database, item utilities, transaction utilities, item investments, and the efficiency threshold. This study builds on HEIM [7] and EFIM-Closed [3], introducing key concepts based on a transaction database where each transaction contains items and their quantities (Table 3).

A transaction database D is a collection of transactions $\{T_1, T_2, \ldots, T_n\}$. Let $I = \{i_1, i_2, \ldots, i_l\}$ be the finite set of all items in the database. Each transaction T_m consists of a list of items $i(T_m) \subseteq I$ and their corresponding quantities $q(i_k, T_m)$.

Table 1. Transaction database

TID	Items	Quantities
T_1	a, b, d, e, f, g	2,2,1,3,2,1
T_2	b, c	1,5
T_3	b, c, d, e, f	2,1,3,2,1
T_4	c, d, e	2,1,3
T_5	a, f	2,3
T_6	a, b, c, d, e, f, g	2,1,4,2,1,3,1
T_7	b, c, e	3,2,2

Let the utility of item i_k be $p(i_k)$. Its utility in transaction T_m is $u(i_k, T_m) = p(i_k) \times q(i_k, T_m)$. For an itemset X, its utility in T_m is $u(X, T_m) = \sum_{i_k \in X} u(i_k, T_m)$. And its total utility in the database is $u(X) = \sum_{X \subseteq T_m} u(X, T_m)$.

Table 2. Unit profit of each item in the database

Item	a	b	c	d	e	f	g
Profit	2	3	1	4	1	5	2

Table 3. Unit investment and Total investment of each item in the database

Item	a	b	c	d	e	f	g
Unit investment	11	8	4	12	3	11	20
Total investment	66	72	56	84	33	99	40

Each item i_k has a unit investment value $uinve(i_k)$. Its total investment in the database is $inve(i_k) = \sum_{i_k \in T_m} uinve(i_k) \times q(i_k, T_m)$. For an itemset X, the total investment is $inve(X) = \sum_{i_k \in X} inve(i_k)$.

Definition 1. Search Space [16] **.** HEIM's search space is modelled as a Set Enumeration Tree, which lists all possible itemsets from the itemset I of the database. The algorithm uses Depth-First Search (DFS) and prunes unpromising itemsets and their supersets based on upper bounds satisfying the DCP.

Definition 2. Transaction Utility – TU [10] **.** Utility of a transaction T_m, denoted as $TU(T_m)$, is defined as: $TU(T_m) = \sum_{i_k \in T_m} u(i_k, T_m)$.

Definition 3. Transaction Weighted Utility – TWU [10] **.** The transaction weighted utility (TWU) of an item i_k in a database, denoted as $TWU(i_k)$, is defined as $TWU(i_k) = \sum_{i_k \in T_m} TU(T_m)$.

In the database shown in Table 1, with unit utility provided in Table 2, the utility of each transaction is presented in Table 4, and the transaction-weighted utility of each item is shown in Table 5.

Definition 4. Sort the order of Items and sort Transactions [10] **.** Let I be the set of all items in the database. An item v is said to precede item w (denoted $v \prec w$) if $TWU(v) \leq TWU(w)$. Based on the database in Tables 1 and 2 and the TWU value in Table 5, the items are ordered as $g \prec a \prec c \prec d \prec f \prec b \prec e$.

Figure 1 provides an example of a Set Enumeration Tree with $I = \{g, a, c, d, f, b, e\}$.

To improve transaction merging, HEPM [6] proposes arranging projected transactions so similar ones are adjacent. Transactions are compared right to left; matching items continue the comparison, otherwise ordering follows item order. Shorter transactions rank lower if comparison ends early. Table 6 illustrates this after sorting items by ascending TWU and ordering transactions.

Table 4. The transaction utility of the database

TID	Items	Quantities	Unit Profits	TU
T_1	a, b, d, e, f, g	2,2,1,3,2,1	2,3,4,1,5,2	29
T_2	b, c	1,5	3,2	8
T_3	b, c, d, e, f	2,1,3,2,1	3,1,4,1,5	26
T_4	c, d, e	2,1,3	1,4,1	9
T_5	a, f	2,3	2,5	19
T_6	a, b, c, d, e, f, g	2,1,4,2,1,3,1	2,3,1,4,1,5,2	37
T_7	b, c, e	3,2,2	3,1,1	13

Table 5. *TWU* of each item in the database

Item	a	b	c	d	e	f	g
TWU	85	113	93	101	114	111	66

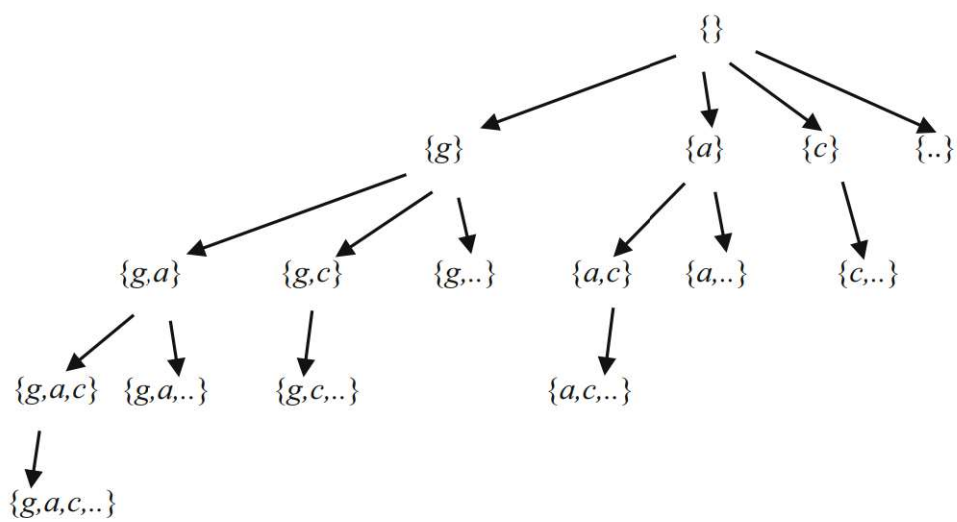

Fig. 1. Set Enumeration Tree with $I = \{g, a, c, d, f, b, e\}$

Definition 5. Efficiency of Itemset [6] **.** The efficiency of an itemset X in the database, denoted $ef(X)$, is defined as: $ef(X) = \frac{u(X)}{inve(X)}$.

Definition 6. Extension Items [16] **.** The set of items that can be used to extend X is defined as:$Ex(X) = \{v \in I | w \prec v, \forall w \in X\}$.

Definition 7. Transaction Weight Efficiency – TWE [6] **.** Transaction Weight Efficiency (TWE) of itemset X, denoted by $TWE(X)$ and is calculated by the formula:$TWE(X) = \frac{\sum_{X \subseteq T_m} TU(T_m)}{inve(X)}$.

Table 6. Database after sorting items and transactions

TID	Items	Profits
T_3	c, d, f, b, e	1,12,5,6,2
T_6	g, a, c, d, f, b, e	2,4,4,8,15,3,1
T_1	g, a, d, f, b, e	2,4,4,10,6,3
T_7	c, b, e	2,9,2
T_4	c, d, e	2,4,3
T_2	c, b	5,3
T_5	a, f	4,15

Property 1. If $TWE(X) < \delta$, then $ef(X) < \delta$. At that time, X and all supersets of X were not HEI.

Property 2. Let $X \subset Y$, we have $TWE(X) \geq TWE(Y)$.

Table 7 shows the TWE of each 1-item in the database.

Table 7. *TWE* of each 1-item in the database

Item	g	a	c	d	f	b	e
TWE	*1.650*	*1.288*	*1.661*	*1.202*	*1.121*	*1.569*	*3.455*

Definition 8. Remaining Utility and Remaining Efficiency [6] **.** Remaining utility of the itemset in the database denoted $ru(X, T_m)$, and is determined as:$ru(X, T_m) = \sum_{i_k \in T_m \wedge i_k \in Ex(X)} u(i_k, T_m)$

Remaining efficiency of itemset in database denoted $ref(X)$ and is calculated as:$ref(X) = \frac{\sum_{X \subseteq T_m} ru(X, T_m)}{inve(X)}$

Property 3. If $ef(X) + ref(X) < \delta$, then all supersets of X are not HEI.

Definition 9. Stricter Upper Bounds [7] **.** The stricter local efficiency of the itemset X with an item v is denoted by $slef(X, v)$ and is defined by the formula:

$$slef(X, v) = \sum_{\{X \cup v\} \subseteq T_m} \left[\frac{u(X, T_m) + u(v, T_m)}{inve(X \cup v)} + \frac{\sum_{i_k \in T_m \wedge i_k \in \{Ex(X) \setminus v\}} u(i_k, T_m)}{inve(X \cup v) + mininv(X \cup v, T_m)} \right]$$

Stricter sub-tree efficiency of the itemset X with an item v is denoted by $ssef(X, v)$ and is defined by the formula:

$$ssef(X, v) = \sum_{\{X \cup v\} \subseteq T_m} \left[\frac{u(X, T_m) + u(v, T_m)}{inve(X \cup v)} + \frac{ru(v, T_m)}{inve(X \cup v) + mininv(X \cup v, T_m)} \right]$$

Property 4. Let itemset X with $\forall v \in Ex(X)$, then $slef(X, v) \geq ssef(X \cup v) \geq ef(X \cup v)$. With the database in Table 6 and $X = \{g\}$, Table 8 presents the results of computing the upper bound of the set $\{g\}$ with items.

Table 8. The result of computing the upper bounds of the set $\{g\}$ with items

Items	a	c	d	f	b	e
ssef	0.5017	0.2718	0.3711	0.2842	0.1437	0.1096
slef	0.5017	0.3028	0.4475	0.4237	0.4816	0.9041
ef	0.1132	0.0625	0.129	0.2086	0.116	0.1096

Definition 10. Primary item list and secondary item list of an itemset.
X[7]**.** To determine the items that can be extended from an itemset X, two lists are used: the primary items ($Pri(X)$) and the secondary items ($Sec(X)$), are defined as follows:

$$Pri(X) = \{i_k | i_k \in I \wedge ssef(X, i_k) \geq \delta\}$$

$$Sec(X) = \{i_k | i_k \in I \wedge sef(X, i_k) \geq \delta\}$$

Definition 11. Closed Itemset [2] **.** Given an itemset X in a transaction database D, the closure operator $c(X)$ is defined as the set of all items that appear in every transaction containing X. An itemset X is called closed if $c(X) = X$. In other words, X is closed when there does not exist any itemset Y such that $X \subset Y$ and $support(Y) = support(X)$.

Definition 12. Forward Extension/Backward Extension [3] **.** Given an itemset $Pe = X \cup i$. Pe is said to have a forward extension if there exists an item z such that $i \prec z$, $z \in Ex(Pe)$, and $support(Pe \cup z) = support(Pe)$. Pe is said to have a backward extension if there exists an item z such that $z \prec i$, $z \notin Ex(Pe)$, and $support(Pe \cup z) = support(Pe)$.

Property 5. Identifying Closed High-Efficiency Itemsets (CHEI) [3] **.** An itemset $Pe = X \cup i$ is considered a CHEI if it is a HEI and has no forward or backward extensions.

Property 6. Pruning by Backward Extension [3] **.** The entire subtree of an itemset $Pe = X \cup i$ can be pruned from the depth-first search if Pe has a backward extension.

Property 7. Closure Jumping [3] **.** Given an itemset X and its projected database. If $sup(X) = sup(X \cup \{z\})$, $\forall z \in Ex(X)$ then the itemset $X \cup Ex(X)$ is the only closed itemset in the subtree rooted at X. In this case, the entire subtree of X can be pruned, and $X \cup Ex(X)$ is returned if it is a HEI.

4 Proposed Algorithm

Currently, there is no research specifically addressing the problem of mining Closed High-Efficiency Itemsets (CHEIs). Solving this problem helps address a significant limitation of HEI mining algorithms, the large and redundant number of extracted itemsets, which complicates post-processing in data mining. CHEIs are condensed, generalized representations of HEIs that still retain complete utility information.

In this study, the author proposes a CHEI mining algorithm called MHEIClosed. The problem is formulated as follows:

Given a transaction database and a user-defined minimum efficiency threshold δ, the task is to find all itemsets that are both high-efficiency (efficiency $\geq \delta$) and closed.

In the proposed algorithm, the author adopts advanced storage and pruning techniques from both closed itemset mining and high efficiency itemset mining.

4.1 The MHEIClosed Algorithm

The MHEIClosed algorithm begins with an empty itemset X and scans the database to compute TWU values. Items and transactions are then sorted by ascending TWU, and irrelevant or empty transactions are pruned. This reordering enhances projection and transaction merging. The algorithm then computes $ssef(X, i)$ and $CID(X, i)$, constructs the Primary Item List $Pri(X)$, and recursively mines for Closed High-Efficiency Itemsets (CHEIs).

The MHEIClosed algorithm is shown below:

Although these steps improve efficiency, the algorithm may still incur heavy computational load on systems with limited resources, particularly when δ is low.

Output: Set of closed high-efficiency itemsets.

```
1. X ← Ø
2. Scan the database to compute TWU(i) for each item i ∈ I
3. Sec(X) ← {i ∈ I|TWE(i) ≥ δ}
4. Sort the items in ascending order of TWU
5. Reorder items, remove those not in Sec(X) and
   drop empty transactions to obtain projected database D'
6. Sort the transaction in D'
7. Scan D' to compute ssef(X,i) and CID(X,i) for i ∈ Sec(X)
8. Pri(X) ← {i ∈ Sec(X)|ssef(X,i) ≥ δ}
9. Explore(X, D', Pri(X), Sec(X), LCID(X), δ)
```

Line 1. The initial set of Closed High-Efficiency Itemsets is empty.
Lines 2–3. Compute TWU and TWE and and derive $Sec(X)$ based on δ.
Line 4. Sort items in ascending order of TWU.
Line 5. Preprocess to reduce database size and item count.
Line 6. Reorder transactions to enhance projection and merging.
Although these steps improve efficiency, the algorithm may still incur heavy computational load on systems with limited resources, particularly when δ is low.
Line 7. Compute $ssef(X,i)$ and $CID(X,i)$ to prune unpromising itemsets.
Line 8. Determine the Primary Item List Pri(X) based on δ.
Line 9. Recursively call Explore to mine CHEIs

4.2 Search Space Exploration

The Explore procedure is called recursively during the mining process to traverse the set-enumeration tree. The input includes the current itemset X being explored, the projected database $D\{X\}$, the Primary Item List $Pri(X)$, the Secondary Item List $Sec(X)$, the list of transaction IDs containing X in the database $LCID(X)$, and the user-defined minimum efficiency threshold δ.

For each item $i \in Pri(X)$, the algorithm extends X with i to form a larger itemset Z. For Z, the algorithm computes its efficiency and the projected database $D\{Z\}$. If $ef(Z) \geq \delta$ then Z is a HEI. The algorithm then checks whether Z has a backward extension. If not, it proceeds to compute $ssef(Z, v)$, $slef(Z, v)$ *and* $CID(Z, v)$ for all $v \in Sec(X)$ where $i \prec v$ and applies the closure jumping technique. If for all $k \neq i$ and $k \in Sec(X)$, the extended itemset $Z \cup \{k\}$ has the same support as Z, then $X \cup Sec(X)$ is the only closed itemset in the current subtree. In this case, the algorithm re-evaluates the efficiency of $X \cup Sec(X)$, and if it satisfies δ, it is a CHEI. If Z has a backward extension, then Z is an HEI but not closed. If Z has no forward extension, then Z is a CHEI. Finally, the algorithm updates the Primary Item List $Pri(Z)$ and Secondary Item List $Sec(Z)$, and recursively calls.

The Explore procedure is presented below:

Input: Set X, $D\{X\}$, $Pri(X)$, $Sec(X)$, $LCID(X)$, δ.
Output: Set of CHEIs extended from X.

```
1  For each i ∈ Pri(X) Do
2    Z ← X ∪ i
3    Compute ef(Z), construct D{Z} (using transaction merging)
4    If (ef(Z) ≥ δ) Then
5      If (Z has no backward extension) Then
6        Scan D{Z} to compute: ssef(Z,v), slef(Z,v), CID(Z,v)
7        If (sup(Z) = sup(Z ∪ k), v≺ k ∀k ∈ Sec(X)) Then
8          Build closed itemset C ← X ∪ Sec(X)
9          Compute ef(C)
10         If ef(C) ≥ δ Then
11           C is a CHEI
12         End If
13       End If
14     Else
15       Pri(Z) = {v ∈ Sec(X) | ssef(Z,v) ≥ δ}
16       Sec(Z) = {v ∈ Sec(X) | slef(Z,v) ≥ δ}
17       Explore(Z,D{Z},Pri(Z),Sec(Z),LCID(Z),δ)
18       If (Z has no forward extension) THEN
19         Z is a CHEI
20       End If
21     End If
22   End If
23 End For
```

Line 1. Iterate over items in $Pri(X)$.
Line 2. For each $i \in Pri(X)$. Extend X to form $Z = X \cup i$.
Line 3. Scan the projected database $D\{X\}$ to compute $ef(Z)$ and construct $D\{Z\}$ using $CID(X, i)$ and $LCID(X)$.
Line 4. Only itemsets with $ef(Z) \geq \delta$ are considered for closure checking.
Line 5. If Z has no backward extension
Line 6. Compute $ssef(Z, v)$, $slef(Z, v)$, and $CID(Z, v)$ for $v \in Sec(X)$, $i \prec v$.
Lines 7–11. Apply closure jumping: if $sup(Z) = sup(Z \cup k)$ for all $k \in Sec(X)$, $k \neq i$, build closed itemset $C = X \cup Sec(X)$, compute $ef(C)$; if $ef(C) \geq \delta$, then C is a CHEI.
Line 14–17. If Z does not satisfy closure jumping, then update $Pri(Z)$, $Sec(Z)$ and recursively call Explore(Z,$D\{Z\}$,$Pri(Z)$,$Sec(Z)$,$LCID(Z)$,δ).
Lines 18–19. Check forward extension: if none, then Z is a CHEI.

5 Experiments

As no prior work tackles Closed High-Efficiency Itemset Mining, the authors adapt the HEPMiner algorithm by adding a closure-checking mechanism, creating HEPMClosed. This serves as the baseline for comparing with the proposed MHEIClosed algorithm.

5.1 Experimental Environment and Database

The evaluation was conducted on a desktop computer running Windows 11, equipped with 16 GB of DDR4 RAM and a 12th Gen Intel® Core™ i7–12700 CPU @ 2.10 GHz. All algorithms used in the experiments were implemented in the Java programming language, specifically using Java version 21.0.6 (LTS), released on January 21, 2025.

Table 9. Databases and their characteristics

Database	Trans	Items	Type
Chainstore	1112,949	46,086	Sparse
Retail	88,162	16,470	Sparse
BMS	59,602	497	Sparse
Pumsb	49,046	2113	Dense

5.2 Comparison and Analysis of Execution Time

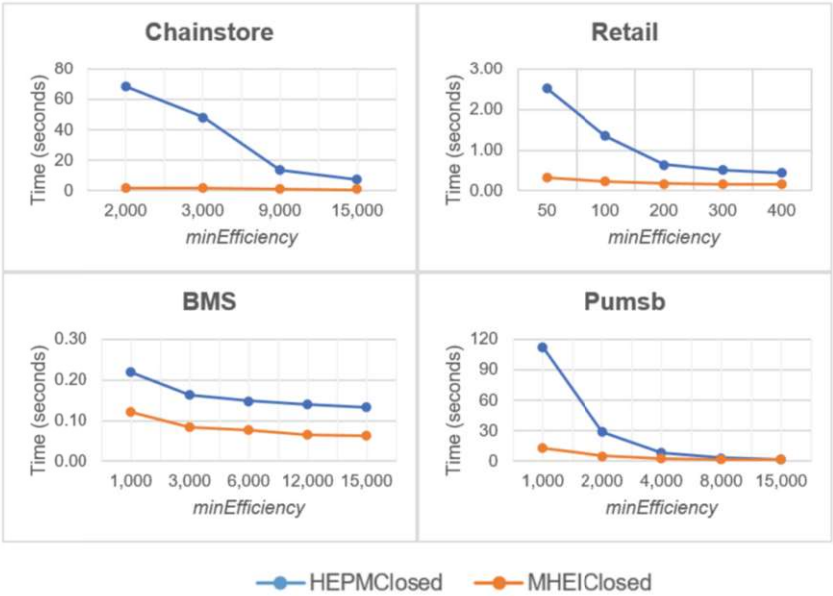

Fig. 2. Comparison of execution time

Figure 2 compares the runtime of HEPMClosed and MHEIClosed across four datasets under different *minEfficiency* thresholds. MHEIClosed consistently outperforms HEPMClosed, especially at lower thresholds. In BMS, it's 1.5–2 times faster, despite both being under 0.25 s. In Chainstore, it achieves a speedup of up to 8 ×, and in Pumsb, the improvement is most notable—reducing runtime from over 100 s to ~ 10 s. Even in Retail, MHEIClosed is 2–3 × faster at low thresholds. These gains highlight its scalability and efficiency, particularly on dense data, due to effective pruning and tight bounds ($ssef$,$slef$).

5.3 Comparison of the Memory Consumption

Figure 3 shows that MHEIClosed consistently uses less memory than HEPMClosed across four datasets (BMS, Chainstore, Pumsb, Retail) under different *minEfficiency* thresholds. In sparse datasets (BMS, Retail), it consumes 2–3 times less memory, while in dense ones (e.g., Pumsb), it reduces usage by 40–50%. Even on smaller datasets, MHEIClosed remains more efficient, especially at low thresholds. This highlights its scalability, thanks to pruning strategies, compact data structures, and tight upper bounds (*ssef*, *slef*).

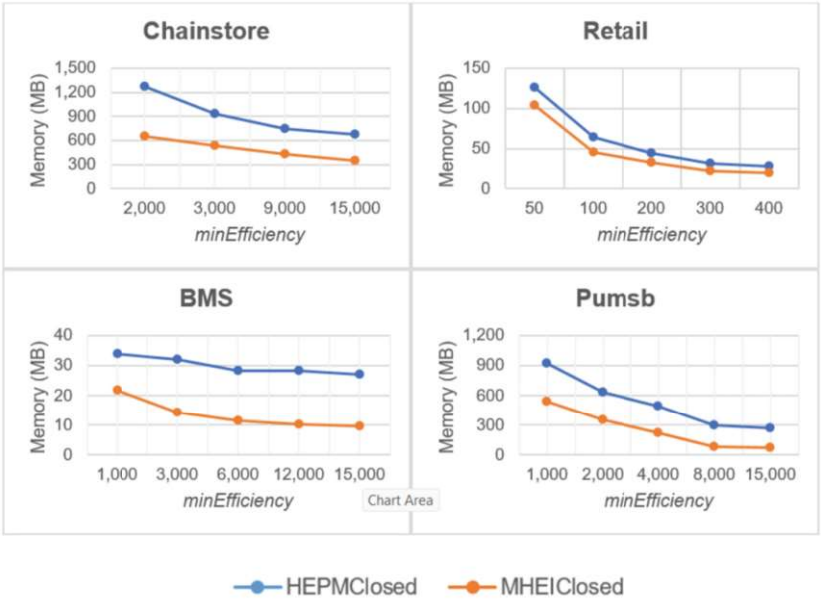

Fig. 3. Comparison of the memory consumption

6 Conclusion and Future Works

This paper presents MHEIClosed, an efficient algorithm for mining Closed High-Efficiency Itemsets (CHEIs) by combining HEIM and closed pattern mining which reduces redundancy while retaining utility and investment. It leverages Efficiency Lists, upper-bound pruning (*ssef*, *slef*), and transaction merging. Experiments on four real datasets demonstrate superior runtime and memory performance compared to HEPM-Closed, particularly for dense data and low thresholds. Future work includes adapting MHEIClosed to handle dynamic/streaming data, as well as applying it to multi-criteria decision-making and cost-sensitive learning.

Acknowledgment. This research is funded by International University, VNU-HCM under grant number SV2022-IT-05.

Disclosure of Interests. The authors have no competing interests to declare that are relevant to the content of this article.

References

1. Yao, H., Hamilton, H.J., Butz, G.J.: A foundational approach to mining itemset utilities from databases. SIAM Proceedings Series (2004)
2. Wu, C.W., Fournier-Viger, P., Gu, J.Y., Tseng, V.S.: Mining closed+ high utility itemsets without candidate generation. In: TAAI 2015 - 2015 Conference on Technologies and Applications of Artificial Intelligence (2016)
3. Fournier-Viger, P., Zida, S., Lin, J.C.W, Wu, C.W., Tseng, V.S.: EFIM-closed: fast and memory efficient discovery of closed high-utility itemsets. In: Perner, P. (eds.) Machine Learning and Data Mining in Pattern Recognition. MLDM 2016. LNCS, vol. 9729. Springer, Cham. https://doi.org/10.1007/978-3-319-41920-6_15
4. Dam, T.L., Li, K., Fournier-Viger, P., Duong, Q. H.: CLS-Miner: efficient and effective closed high-utility itemset mining. Front. Comput. Sci. **13**(2) (2019)
5. Nguyen, T.D.D., Nguyen, L.T.T., Vu, L., Vo, B., Pedrycz, W.: Efficient algorithms for mining closed high utility itemsets in dynamic profit databases. Expert Syst. Appl. **186** (2021)
6. Zhang, X., Chen, G., Song, L., Gan, W., Song, Y: .HEPM: High-efficiency pattern mining. Knowledge-Based Syst. **281** (2023)
7. Huynh, B., et al.: An efficient strategy for mining high-efficiency itemsets in quantitative databases. Knowl.-Based Syst. **299**, 112035 (2024)
8. Agrawal, R., Srikant, R.: Fast algorithms for mining association rules. In: Proc. of 20th International Conference on Very Large Data Bases, {VLDB 1994} (1994)
9. Han, J., Pei, J., Yin, Y.: Mining frequent patterns without candidate generation. SIGMOD Rec. (ACM Spec. Interes. Gr. Manag. Data) **29**(2) (2000)
10. Liu, Y., Liao, Wk., Choudhary, A.: A two-phase algorithm for fast discovery of high utility itemsets. In: Ho, T.B., Cheung, D., Liu, H. (eds.) Advances in Knowledge Discovery and Data Mining. PAKDD 2005. LNCS, vol. 3518. Springer, Berlin, Heidelberg (2005). https://doi.org/10.1007/11430919_79
11. Liu, M., Qu, J.: Mining high utility itemsets without candidate generation categories and subject descriptors. In: Proc. 21st ACM Int. Conf. Inf. Knowl. Manag. (2012)
12. Fournier-Viger, P., Wu, CW., Zida, S., Tseng, V.S. FHM: faster high-utility itemset mining using estimated utility co-occurrence pruning. In: Andreasen, T., Christiansen, H., Cubero, JC., Raś, Z.W. (eds.) Foundations of Intelligent Systems. ISMIS 2014. LNCS, vol 8502. Springer, Cham (2014). https://doi.org/10.1007/978-3-319-08326-1_9
13. Krishnamoorthy, S.: Pruning strategies for mining high utility itemsets. Expert Syst. Appl. **42**(5), 2371–2381 (2015)
14. Duong, Q.-H., Fournier-Viger, P., Ramampiaro, H., Nørvåg, K., Dam, T.-L.: Efficient high utility itemset mining using buffered utility-lists. Appl. Intell. **48**(7), 1859–1877 (2018)
15. Krishnamoorthy, S.: HMiner: Efficiently mining high utility itemsets. Expert Syst. Appl. **90**, 168–183 (2017)
16. Zida, S., Fournier-Viger, P., Lin, J.C.-W., Wu, C.-W., Tseng, V.S.: EFIM: a fast and memory efficient algorithm for high-utility itemset mining. Knowl. Inf. Syst. **51**(2), 595–625 (2017)
17. Nguyen, L.T.T., Nguyen, P., Nguyen, T.D.D., Vo, B., Fournier-Viger, P., Tseng, V.S.: Mining high-utility itemsets in dynamic profit databases. Knowl.-Based Syst. **175**, 130–144 (2019)